

Lecture 11: DNN Interpretability (Explainability)---Input Interpretability

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

Slides on [Axiomatic Attribution for Deep Networks](#) by Sundarajan, Taly, and Yan

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Course logistics

- Reviews:
 - Review due tomorrow at 11:59 PM
 - Paper to review: [Sanity Checks for Saliency Maps](#) by Adebayo et al.
- Projects:
 - Form teams!
 - Enter team members here
 - Proposal format will be out soon
- Non-standard schedule for the next 2 weeks

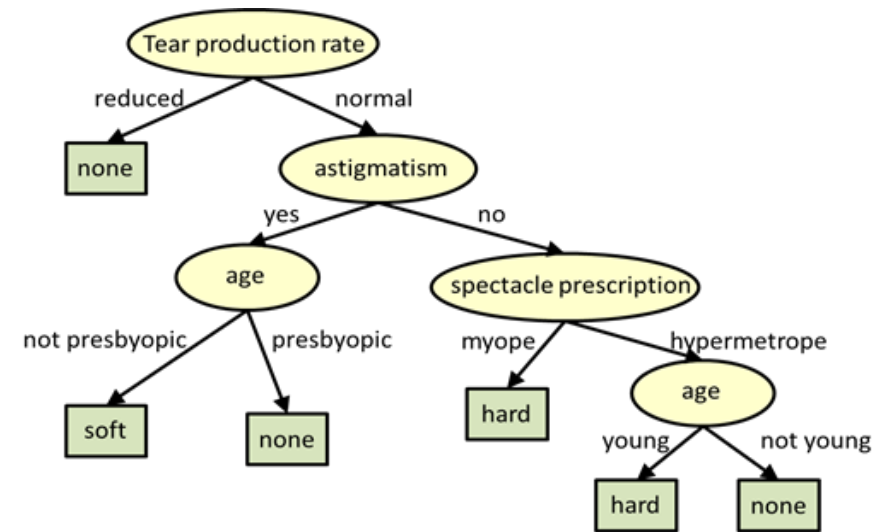
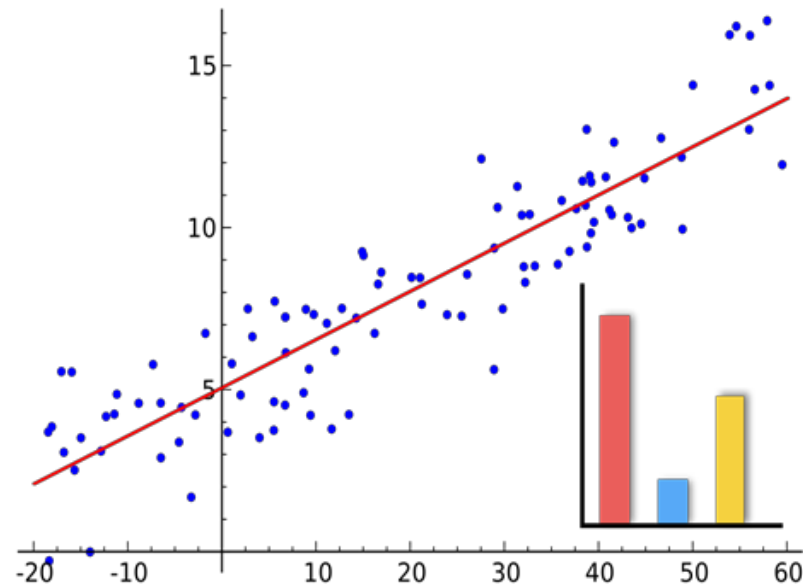
Goals of DNN explainability

- Explain why the model made a particular prediction on a specific input
- Explain how the model makes predictions across all inputs
- Explain how the training data affects model predictions
- Explain what changes to the input can cause the model make a different decision
- ...

Why do we need explanations?

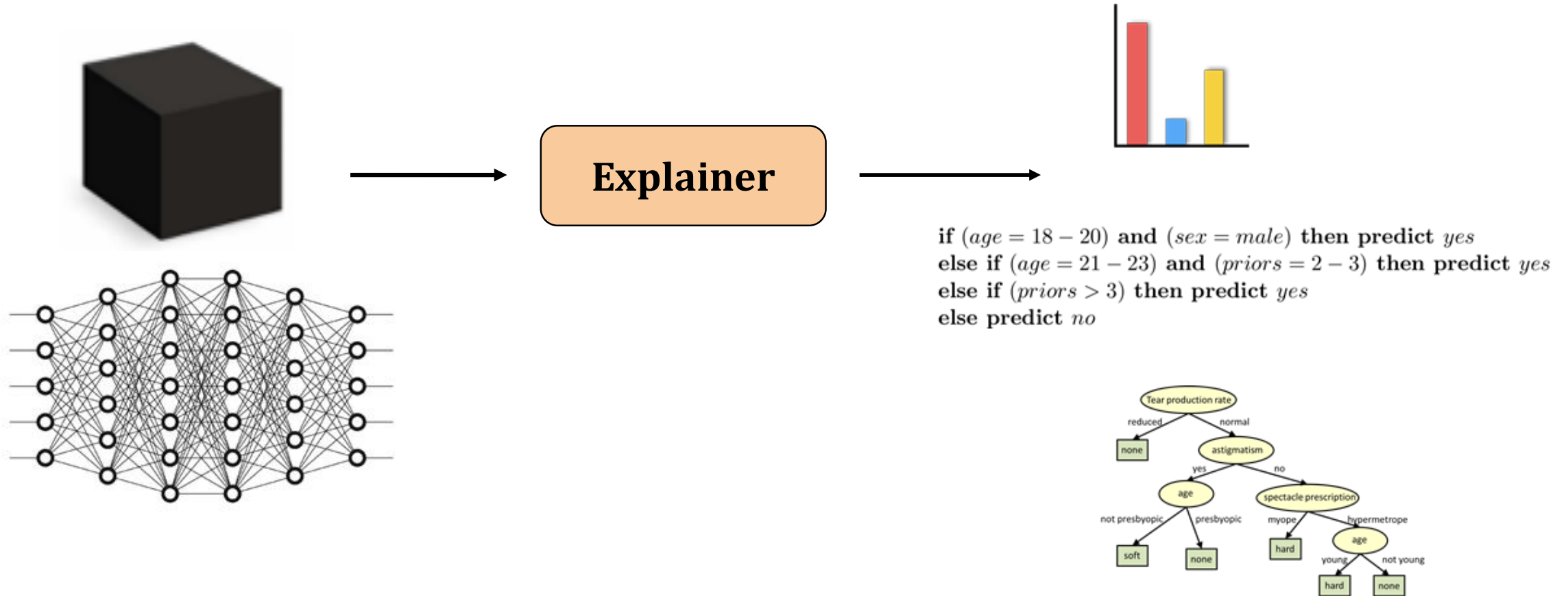
- Explaining predictions facilitates debugging
- Explaining predictions facilitates bias detection
- Explanation helps provide recourse to individuals who are adversely affected by model predictions
- Explanation helps assess if and when to trust model predictions when making decisions
- ...

Inherently interpretable models



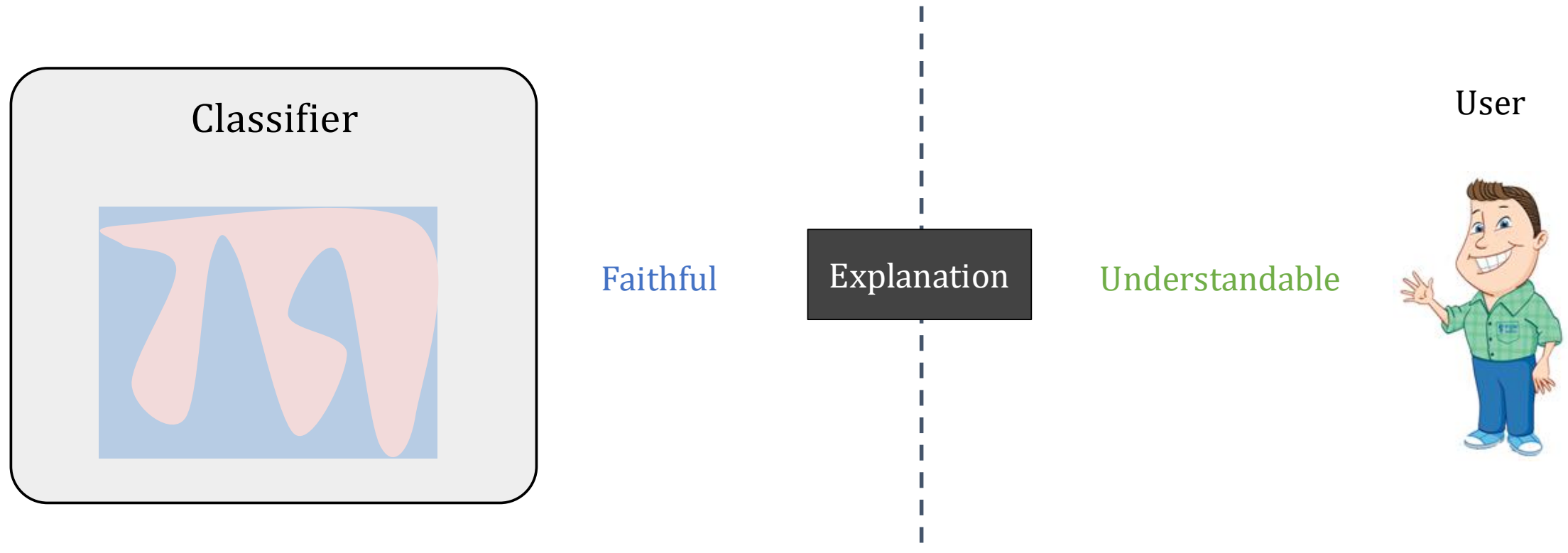
if (*age* = 18 – 20) and (*sex* = male) then predict *yes*
else if (*age* = 21 – 23) and (*priors* = 2 – 3) then predict *yes*
else if (*priors* > 3) then predict *yes*
else predict *no*

Post-hoc explanations



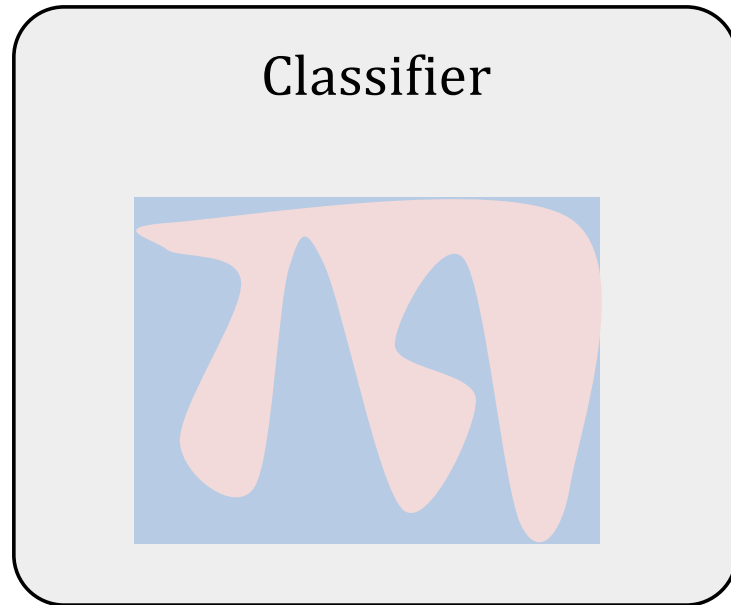
What is an explanation?

Ideally, interpretable description of the model behavior



What is an explanation?

Ideally, interpretable description of the model behavior



Send all the model parameters θ ?

Send many example predictions?

Summarize with a program/rule/tree

Select most important features/points

Describe how to *flip* the model prediction

...

User



Types of post-hoc interpretability techniques

- Input interpretability
 - Feature attribution
 - Influence functions
 - Counterfactual explanations
 - ...
- Inner interpretability
 - Concept-based explanations
 - Mechanistic interpretability
 - ...

Types of post-hoc interpretability techniques

Local vs global explanations

Explain individual predictions

Explain complete behavior of the model

Help unearth biases in the *local neighborhood* of a given instance

Help shed light on *big picture biases* affecting larger subgroups

Help vet if individual predictions are being made for the right reasons

Help vet if the model, at a high level, is suitable for deployment

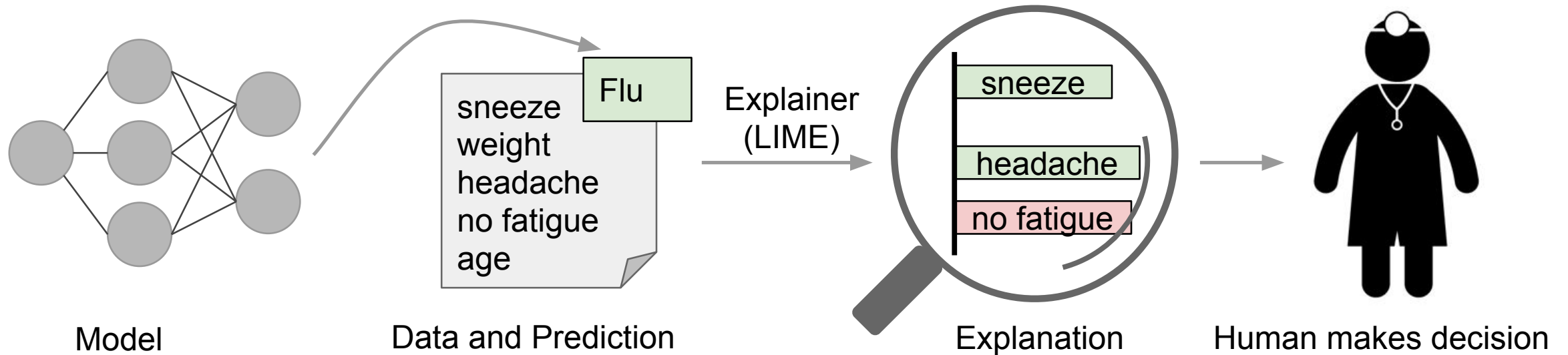
Types of post-hoc interpretability techniques

- Input vs internal interpretability
- Local vs global
- Intrinsic vs post-hoc
- Model-specific vs model-agnostic (White-box vs black-box)
- ...

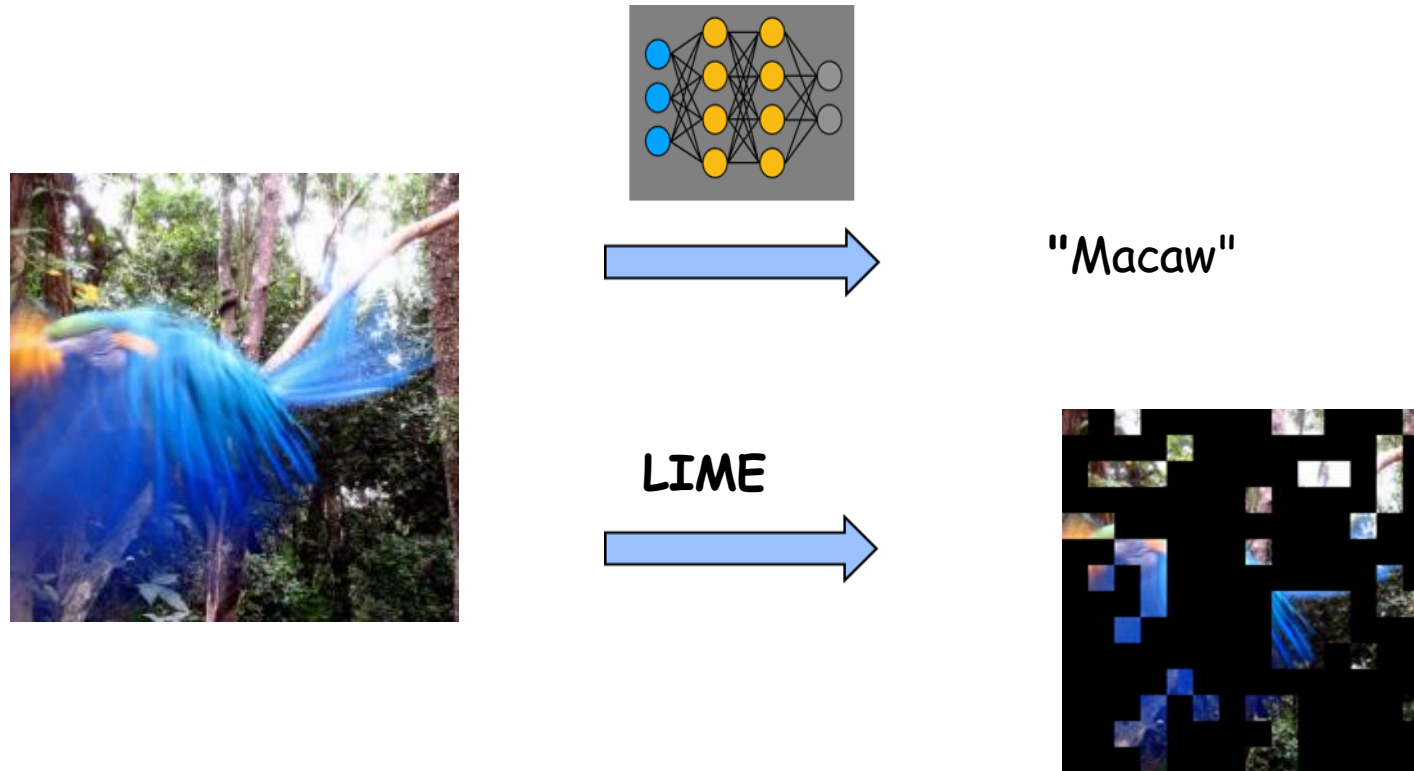
Feature attribution methods

- Goal: Explain why the model made a particular prediction on a specific input
- Solution: Select/rank a subset of input features that contributed the most to model prediction
- Local explanations
- Model-agnostic vs Gradient-based
- Today:
 - **LIME:** Ribeiro et al., “Why should I trust you?” Explaining the predictions of any classifier, KDD’16
 - **Saliency Maps:** Simonyan et al., Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps, ICLR’14
 - **Integrated Gradients:** Sundararajan et al., Axiomatic Attribution for Deep Networks, ICML’17

LIME use-case



LIME explanation for image classification



Classifier: Vision Transformer (Dosovitskiy, 2020)

Dataset: ImageNet

Formalizing the feature attribution problem

- Given an input x and model f , select a subset (of specified size) of features of x that contribute the most to the prediction $f(x)$
- First attempt: if $x \in \mathbb{R}^d$, output should be d -dim vector over $\{0,1\}$ specifying for each feature whether it is selected or not
- Problem: Features in representation of x may not be interpretable to humans
 - e.g. an input image is a tensor with three color channels per pixel

Formalizing the feature attribution problem

- If input x is d -dimensional, first define a simpler d' -dimensional “interpretable” representation
- Output of explanation method g , for given input x and model f , is d' -dim vector over $\{0,1\}$ that selects a subset of interpretable features (possibly with weights)
- Desired properties
 - Model agnostic: Method works for any model f
 - Interpretability: Minimize complexity of g (e.g. select at most 25% features)
 - Local fidelity: g approximates f well in the vicinity of x

Interpretable features: From pixels to superpixels

- Superpixel is a technique to segment an image into regions by considering similarity according to perceptual features
- Segmentation is dependent on specific input
- Well-known algorithms based on graph partitioning and clustering

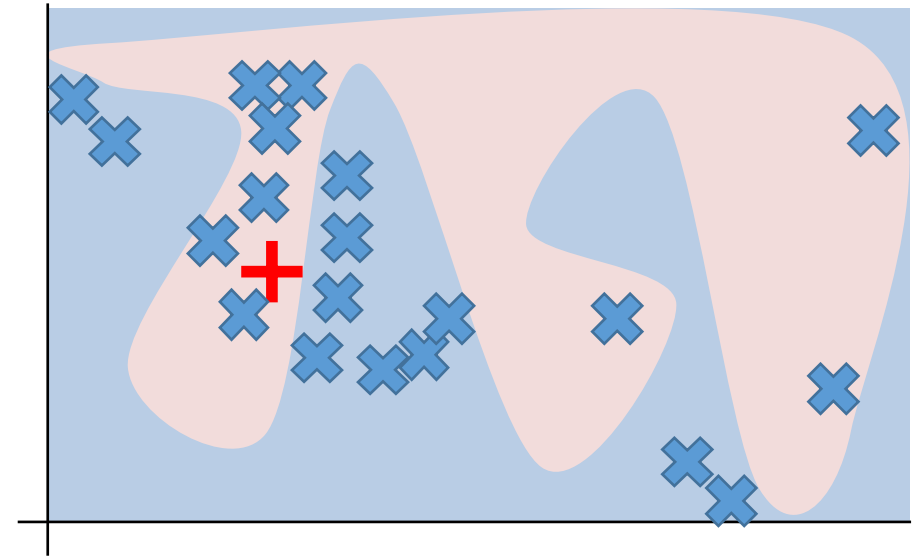
Interpretable features: From pixels to superpixels

Original image



LIME: Local Interpretable Model-agnostic Explanations

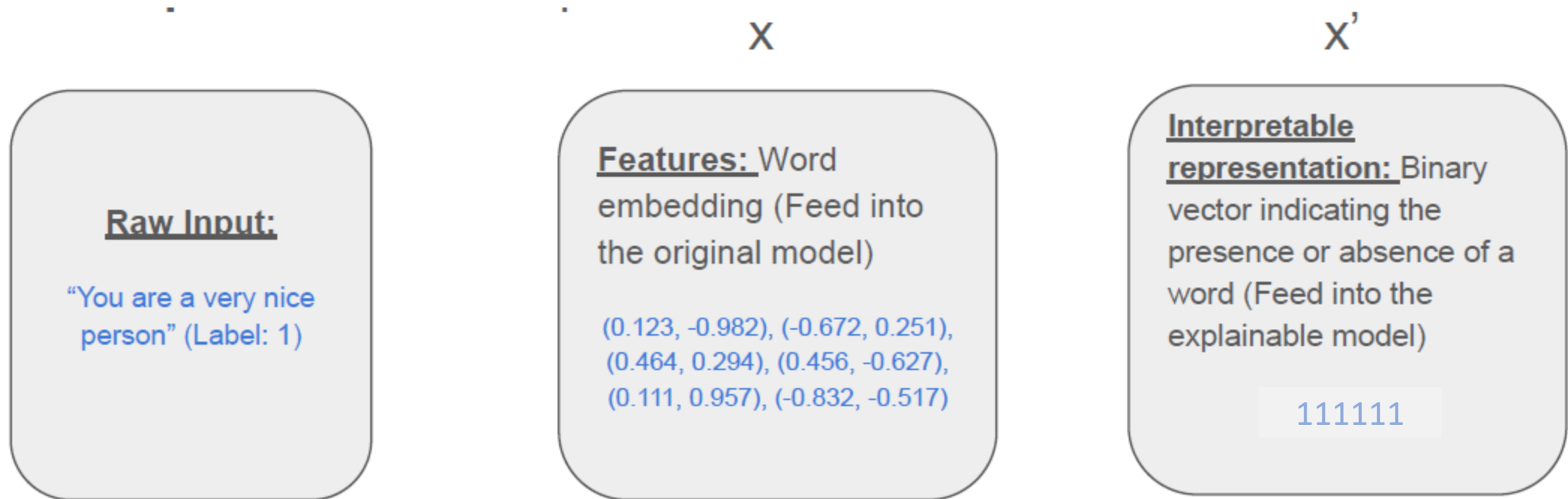
1. Sample points around x
2. Use model to predict labels for each sample
3. Weigh samples according to distance to x
4. Learn simple linear model on weighted samples
5. Use simple linear model to explain



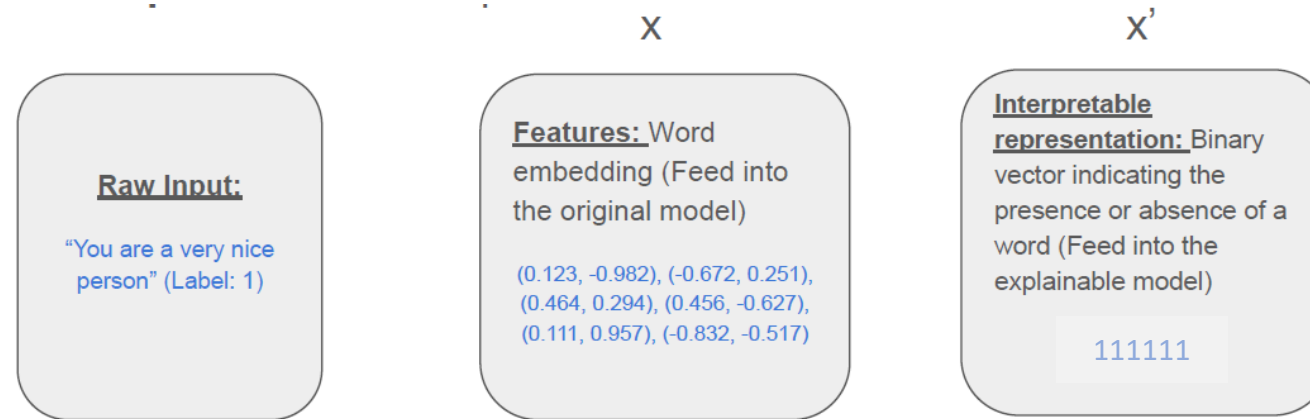
LIME: More details

- Consider a (black-box) classifier that labels an input sentence as good (label 1) or bad (label 0)
- Suppose it labels “You are a very nice person” as 1
- We want as an explanation the 3 words that contributed the most to this prediction

Interpretable data representation

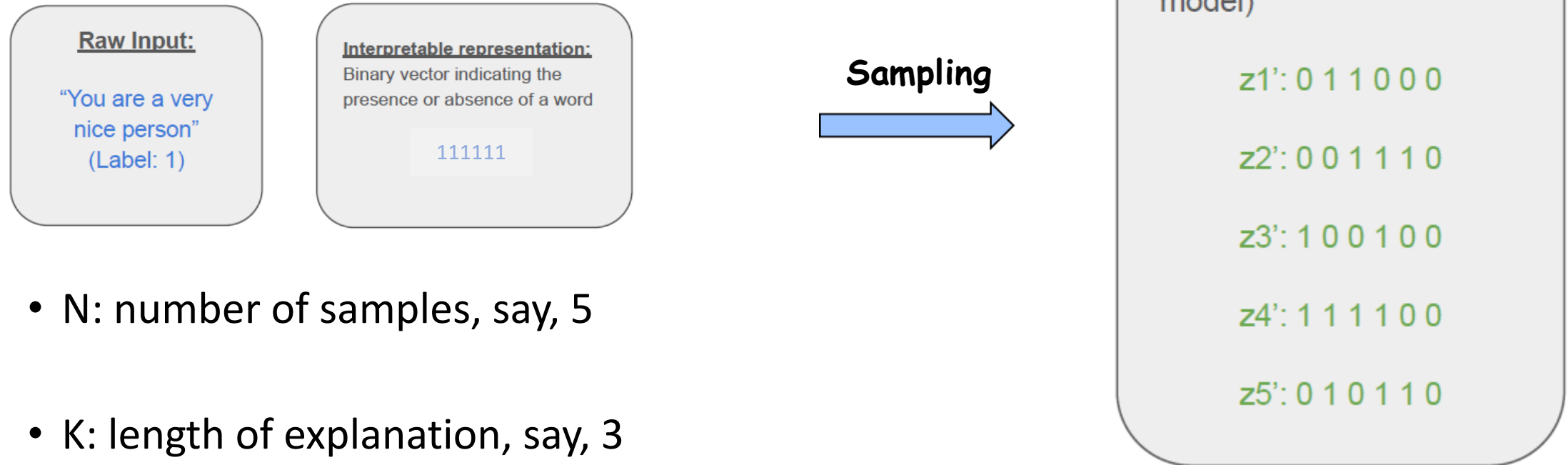


Sampling



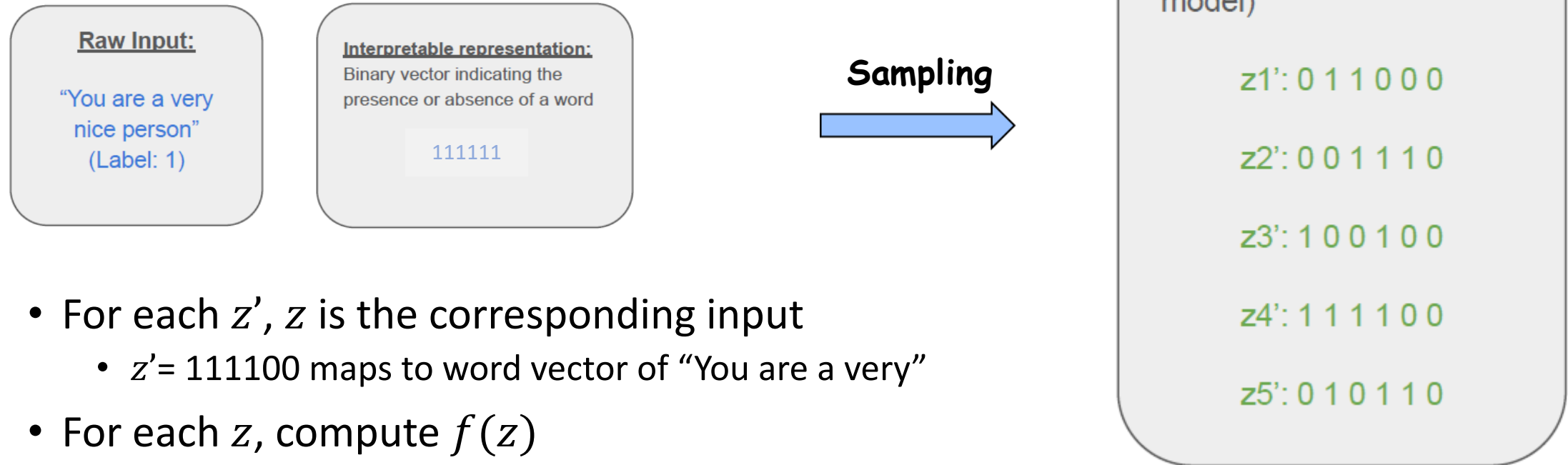
- N: number of samples, say, 5
- K: length of explanation, say, 3
- Sample instances around x' by drawing nonzero elements uniformly at random

Sampling



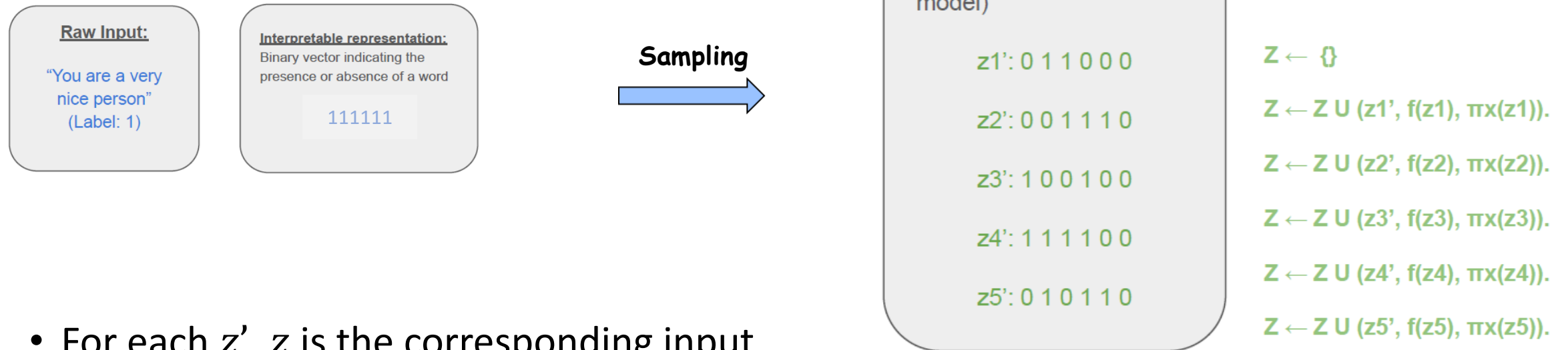
- N: number of samples, say, 5
- K: length of explanation, say, 3
- Sample instances around x' by drawing nonzero elements uniformly at random

Analyzing samples



- For each z' , z is the corresponding input
 - $z' = 111100$ maps to word vector of "You are a very"
- For each z , compute $f(z)$
- $\pi_x(z)$: Proximity measure between an instance z to x

Analyzing samples



- For each z' , z is the corresponding input
 - $z' = 111100$ maps to word vector of "You are a very"
- For each z , compute $f(z)$
- $\pi_x(z)$: Proximity measure between an instance z to x

Finding explainable model

Explainable model g:

Choose linear models here

Dataset:

Z (contains data & label & additional distance metric)

Objective function:

$$\min \underbrace{\mathcal{L}(f, g, \pi_x)}_{\text{Local fidelity}} + \underbrace{\Omega(g)}_{\text{Interpretability}}$$

$\Omega(g)$: A measure of complexity (as opposed to interpretability)

$$\mathcal{L}(f, g, \pi_x) = \sum_{(z', f(z), \pi_x(z)) \in Z} \pi_x(z) (f(z) - g(z'))^2$$

Explanation:

$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

(Corresponding weight for each feature)

Raw Input:

"You are a
very nice
person"
(Label: 1)

Perturbed sample:

Interpretable binary vector
indicating the presence or
absence of a word (Feed
into the explainable model)

z1': 0 1 1 0 0 0

z2': 0 0 1 1 1 0

z3': 1 0 0 1 0 0

z4': 1 1 1 1 0 0

z5': 0 1 0 1 1 0

$Z \leftarrow \{\}$

$Z \leftarrow Z \cup (z1', f(z1), \pi_x(z1)).$

$Z \leftarrow Z \cup (z2', f(z2), \pi_x(z2)).$

$Z \leftarrow Z \cup (z3', f(z3), \pi_x(z3)).$

$Z \leftarrow Z \cup (z4', f(z4), \pi_x(z4)).$

$Z \leftarrow Z \cup (z5', f(z5), \pi_x(z5)).$

Finding explainable model

Explainable model g:

Choose linear models here

Dataset:

Z (contains data & label & additional distance metric)

Objective function:

$$\min \underbrace{\mathcal{L}(f, g, \pi_x)}_{\text{Local fidelity}} + \underbrace{\Omega(g)}_{\text{Interpretability}}$$

$\Omega(g)$: A measure of complexity (as opposed to interpretability)

$$\mathcal{L}(f, g, \pi_x) = \sum_{(z', f(z), \pi_x(z)) \in Z} \pi_x(z) (f(z) - g(z'))^2$$

Explanation:

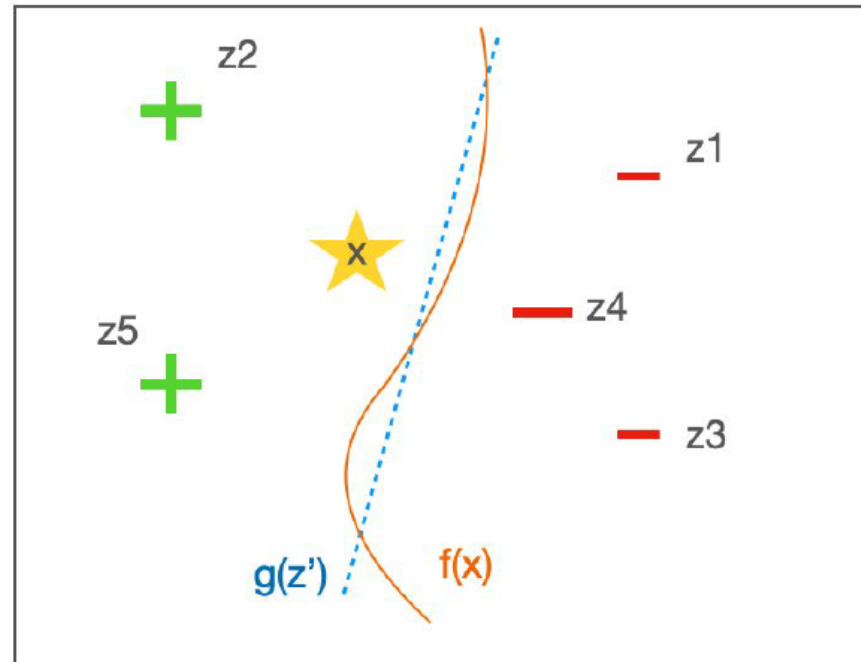
$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

(Corresponding weight for each feature)

For the toy example:

Choose K-Lasso to limit # of explanations (K=3), i.e. we can only choose up to 3 words here for explanation

Explainable model g vs original model f



Explanation:

Nice 0.96
Very 0.56
You 0.43

LIME algorithm

Algorithm 1 Sparse Linear Explanations using LIME

Require: Classifier f , Number of samples N

Require: Instance x , and its interpretable version x'

Require: Similarity kernel π_x , Length of explanation K

$\mathcal{Z} \leftarrow \{\}$

for $i \in \{1, 2, 3, \dots, N\}$ **do**

$z'_i \leftarrow \text{sample_around}(x')$

$\mathcal{Z} \leftarrow \mathcal{Z} \cup \langle z'_i, f(z_i), \pi_x(z_i) \rangle$

end for

$w \leftarrow \text{K-Lasso}(\mathcal{Z}, K)$ $\triangleright$ with z'_i as features, $f(z)$ as target

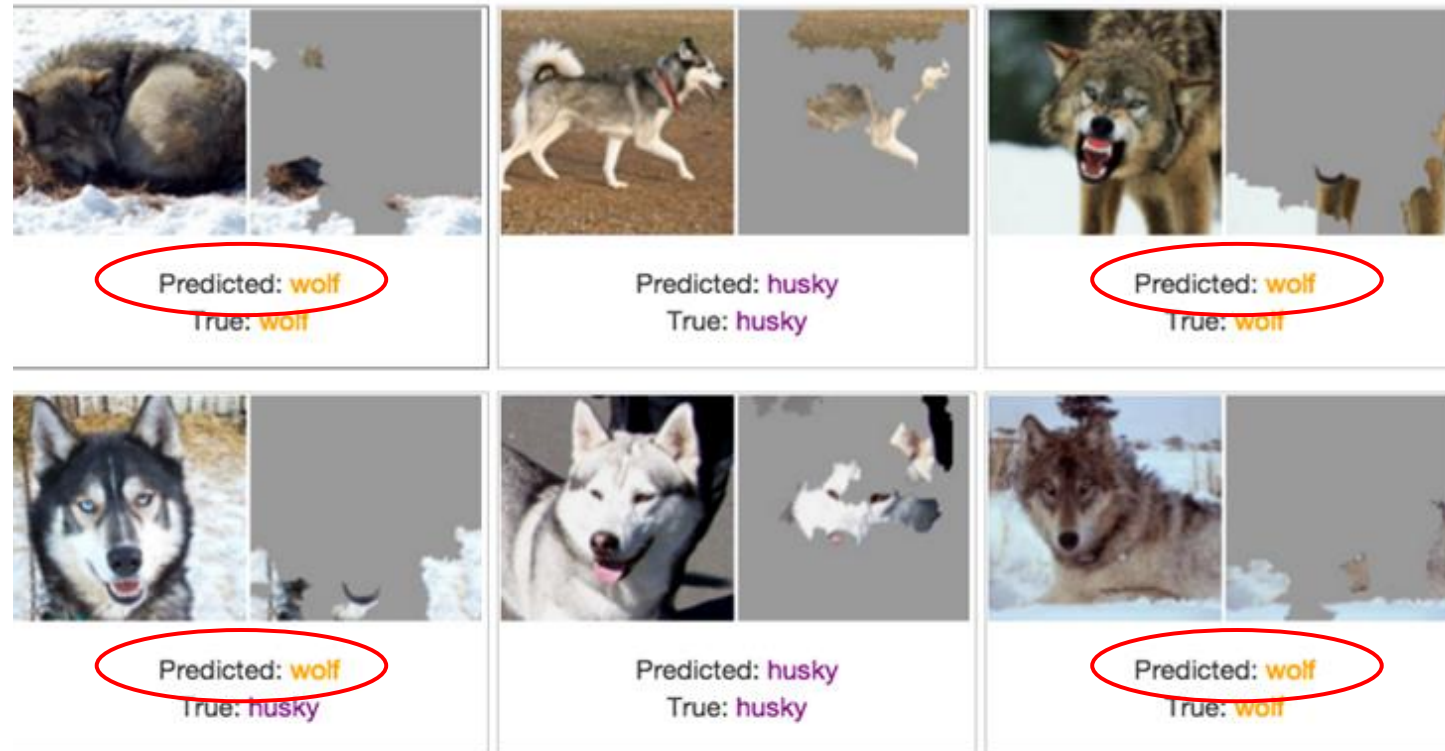
return w

Image classifier: wolf vs husky

Only 1 mistake!

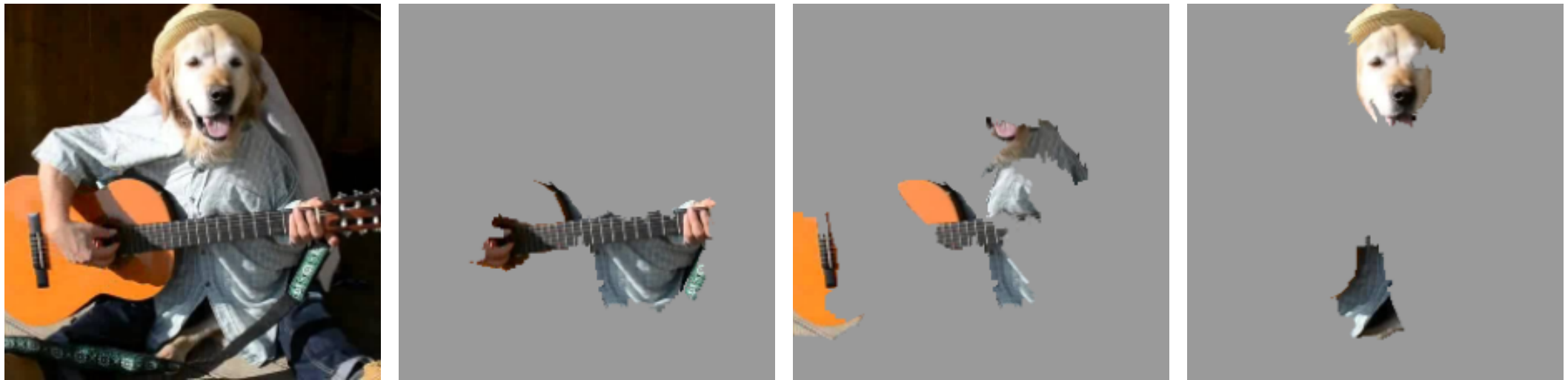


Check explanations with LIME



We've built a great snow detector...

Explanations with LIME



(a) Original Image

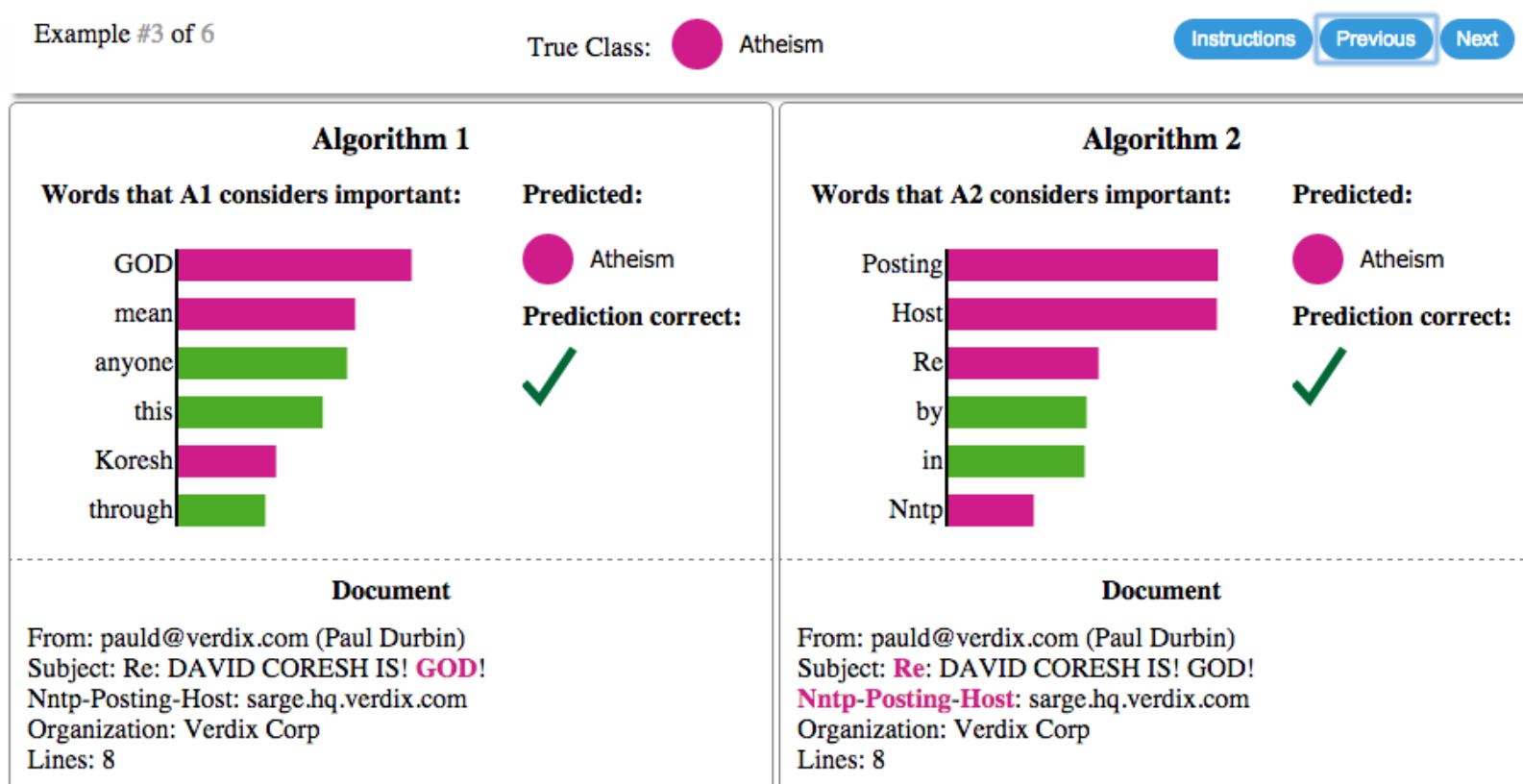
(b) Explaining *Electric guitar*

(c) Explaining *Acoustic guitar*

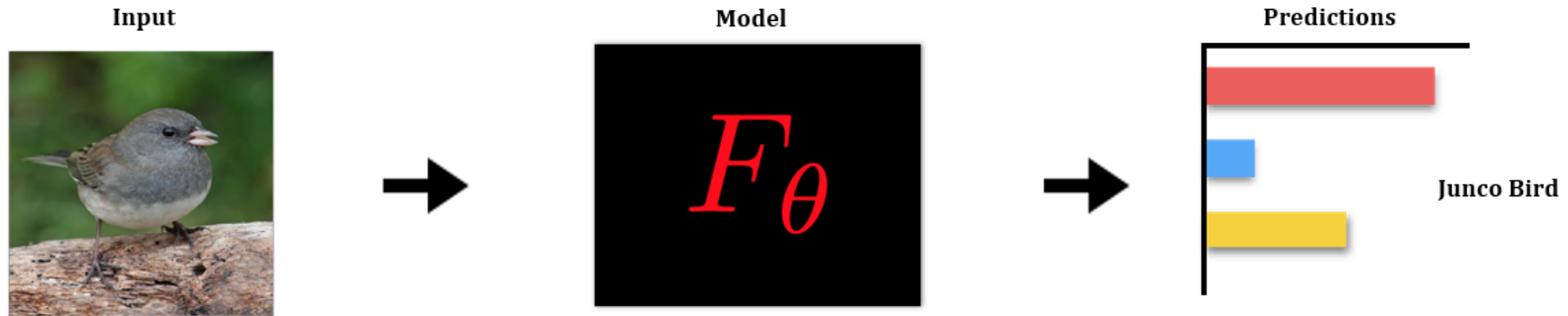
(d) Explaining *Labrador*

Figure 4: Explaining an image classification prediction made by Google’s Inception neural network. The top 3 classes predicted are “Electric Guitar” ($p = 0.32$), “Acoustic guitar” ($p = 0.24$) and “Labrador” ($p = 0.21$)

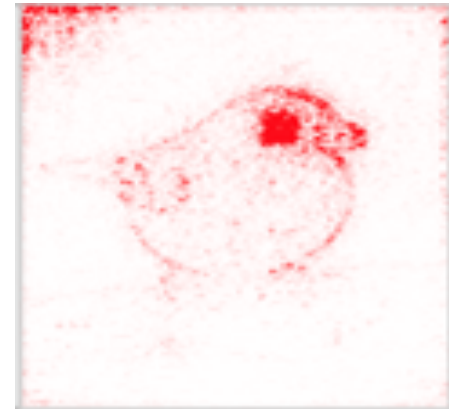
LIME explanations can help choose between models



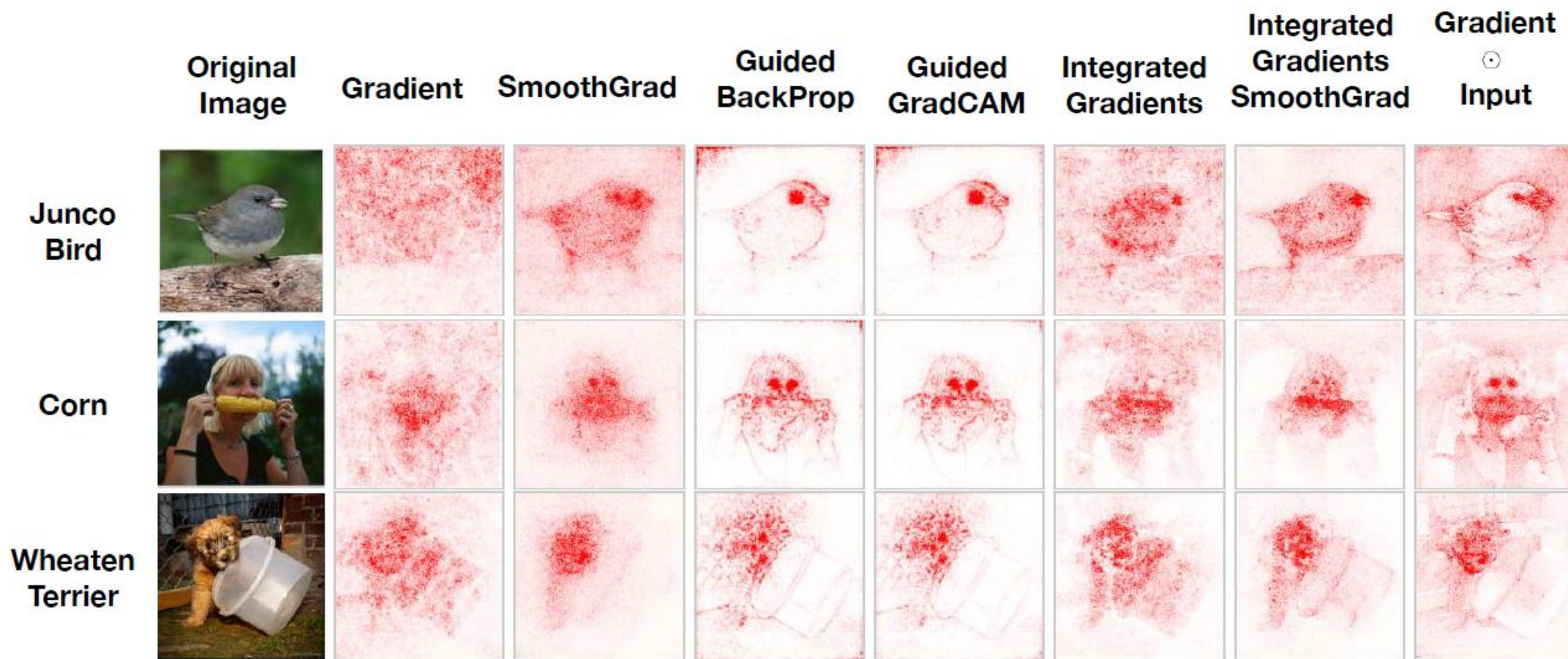
Saliency maps



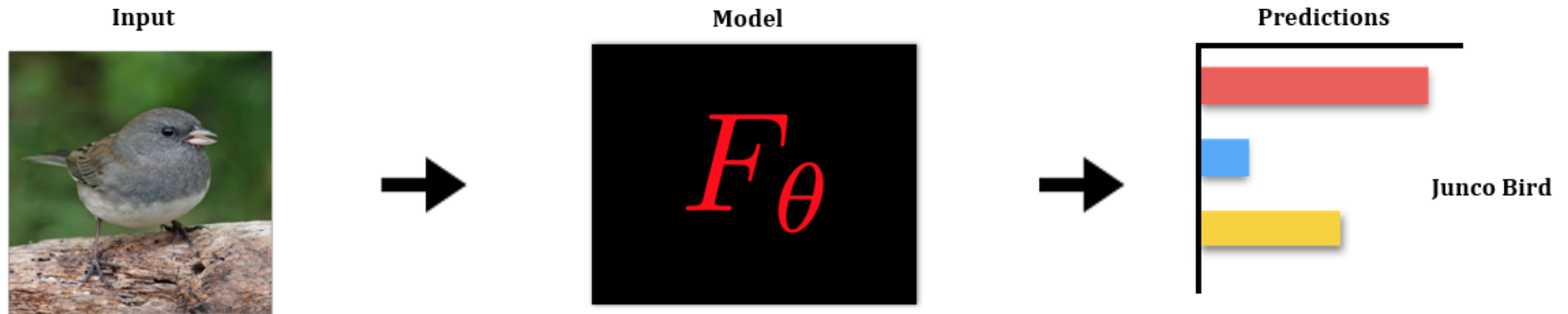
What parts of the input are most relevant for the model's prediction:
'Junco Bird'?



Different kinds of saliency maps



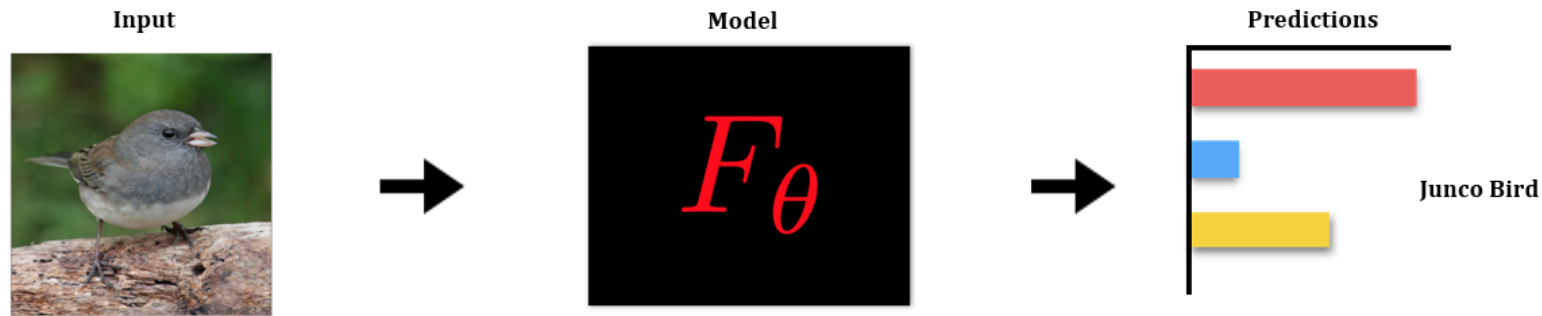
Saliency maps for DNNs



$$F : \mathbb{R}^d \rightarrow \mathbb{R}^c \quad \text{Model}$$

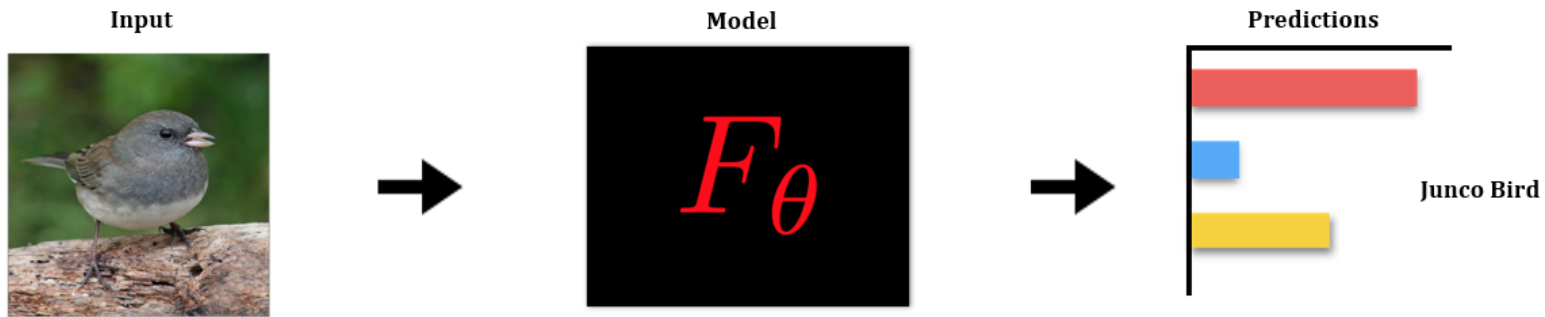
$$F_i : \mathbb{R}^d \rightarrow \mathbb{R} \quad \text{class specific logit}$$

Input gradient



- What's the contribution of a pixel in input image to the prediction "Junco bird"?
- Solution: Compute the gradient of the output logit w.r.t. the pixel

Input gradient



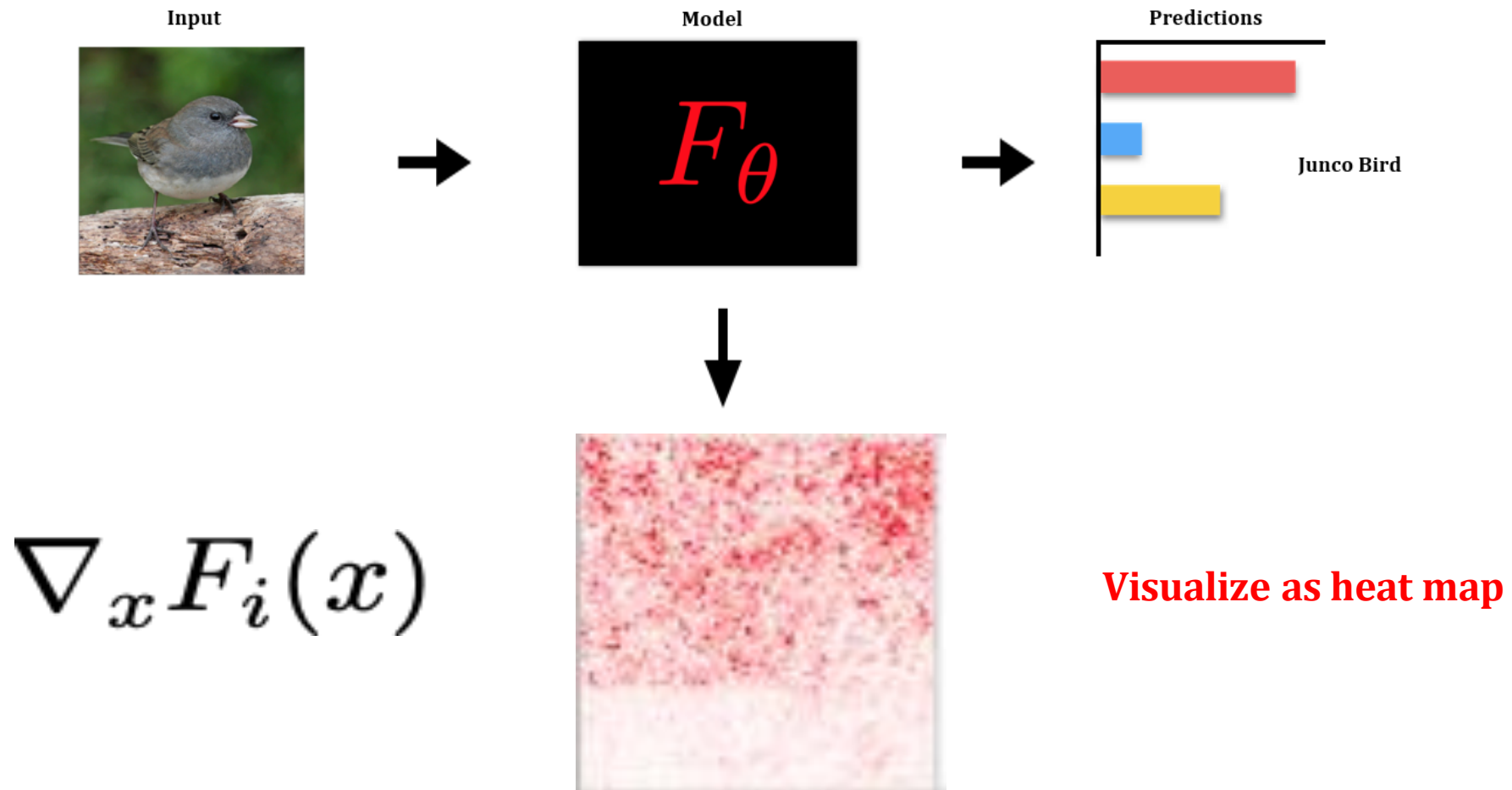
$$\nabla_x F_i(x) \in \mathbb{R}^d$$

Input

Logit

Same dimension as the input

Input gradient

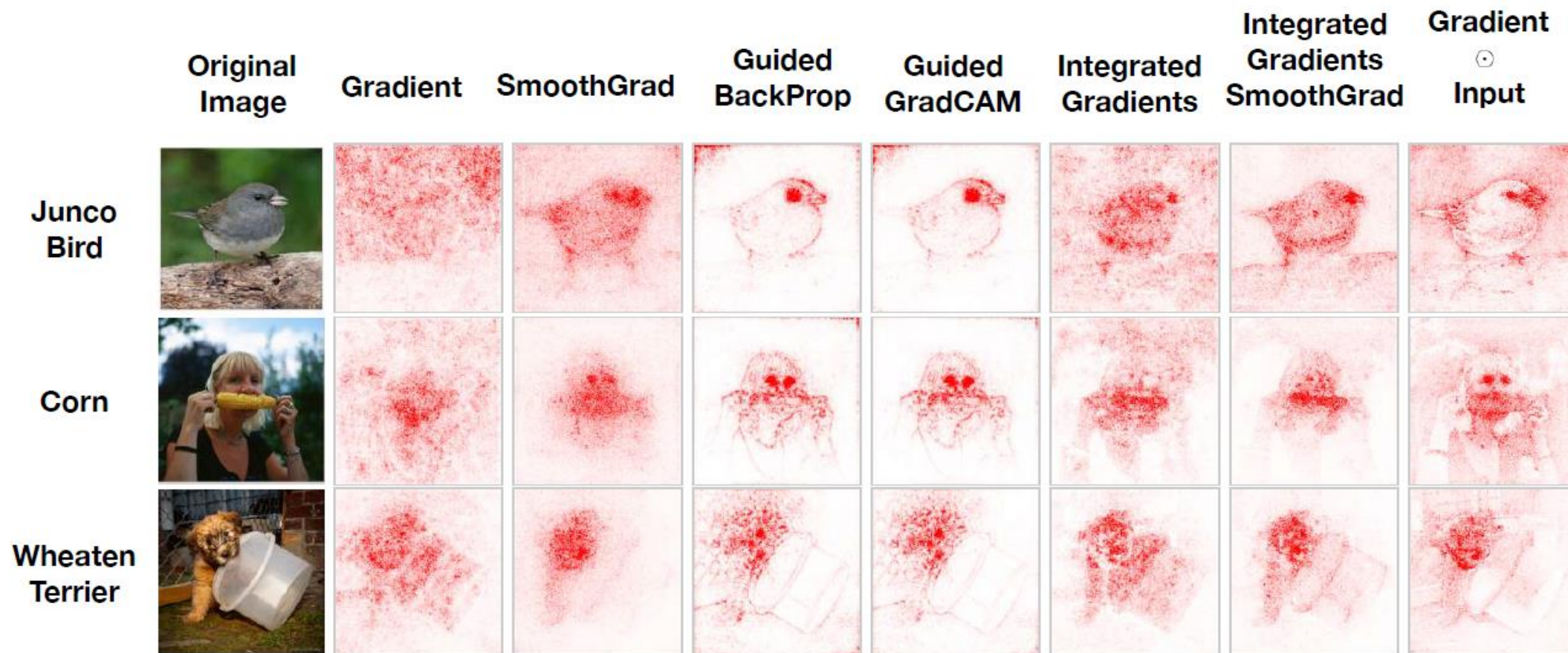


Beyond input gradients

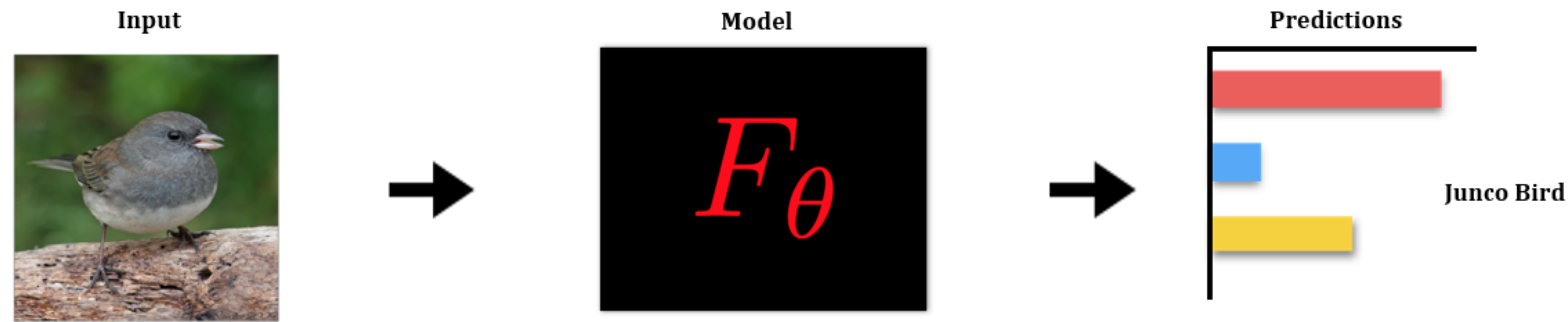


- Input gradients capture sensitivity to individual pixels/features well, but ...raw gradients are visually noisy
- Many improvements proposed in literature

Beyond input gradients



SmoothGrad



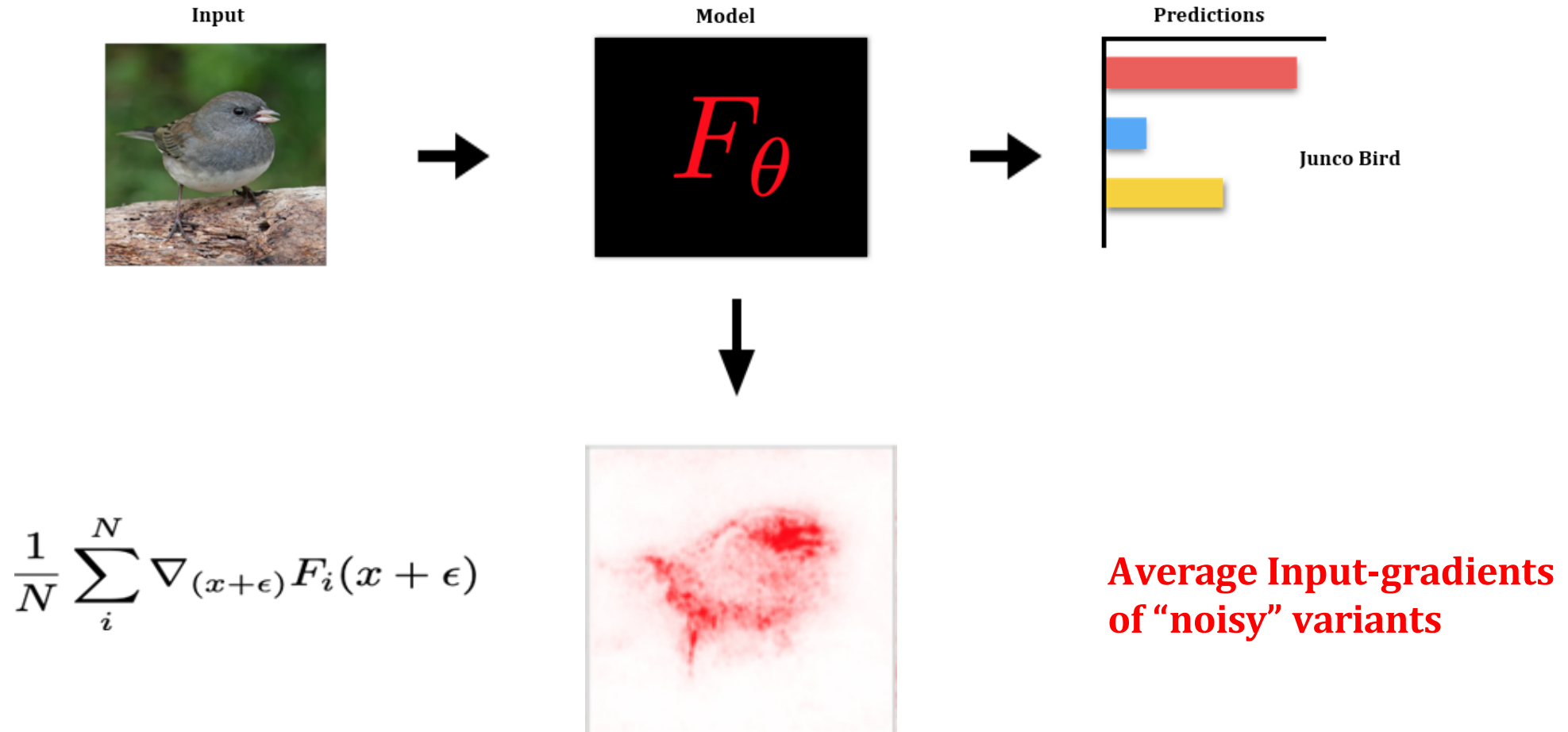
SmoothGrad

$$\frac{1}{N} \sum_i^N \nabla_{(x+\epsilon)} F_i(x + \epsilon)$$

Gaussian noise

- Construct N perturbations of input x
- By adding noise ϵ sampled from Gaussian distribution with standard deviation σ
- For each variant compute input gradient
- Take average

SmoothGrad



SmoothGrad: Effect of noise

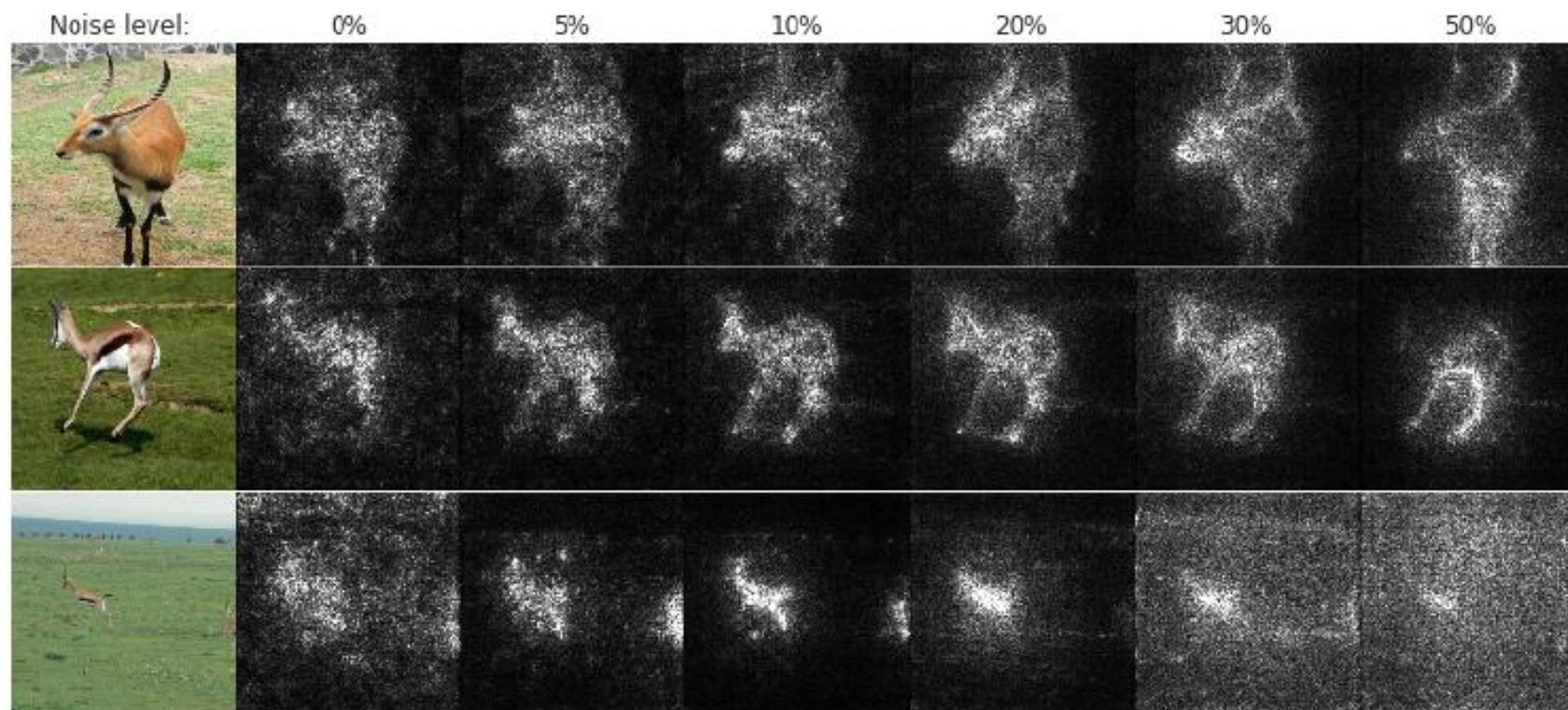
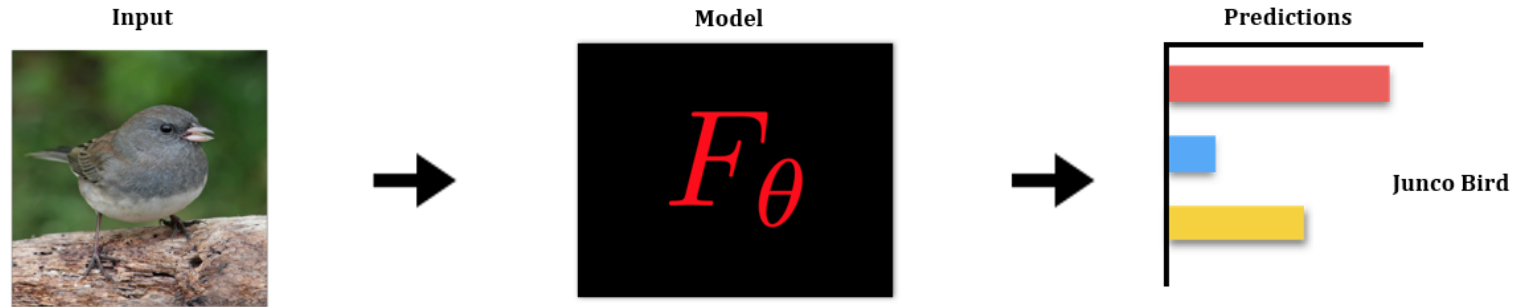


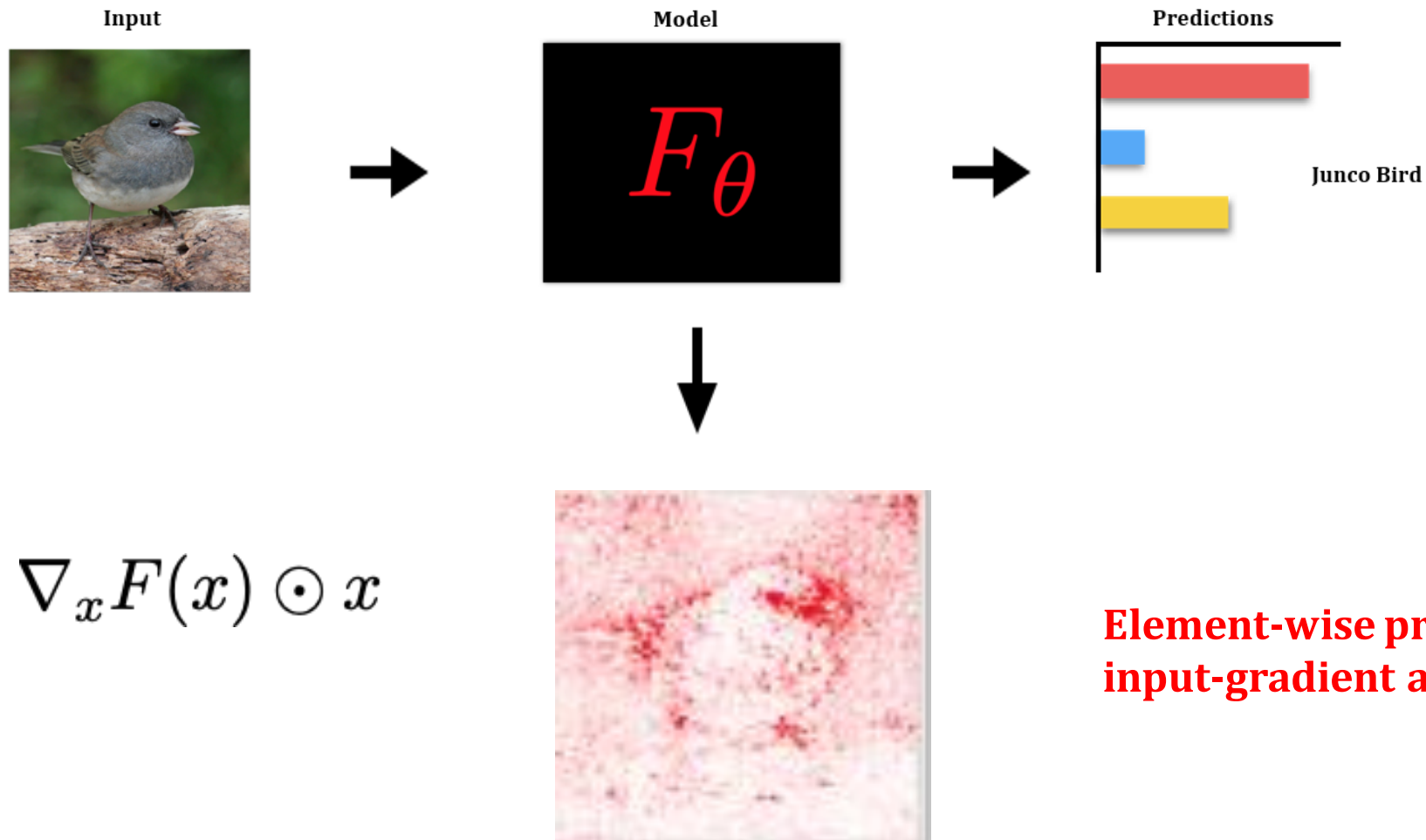
Figure 3. Effect of noise level (columns) on our method for 5 images of the gazelle class in ImageNet (rows). Each sensitivity map is obtained by applying Gaussian noise $\mathcal{N}(0, \sigma^2)$ to the input pixels for 50 samples, and averaging them. The noise level corresponds to $\sigma / (x_{max} - x_{min})$.

Gradient-Input



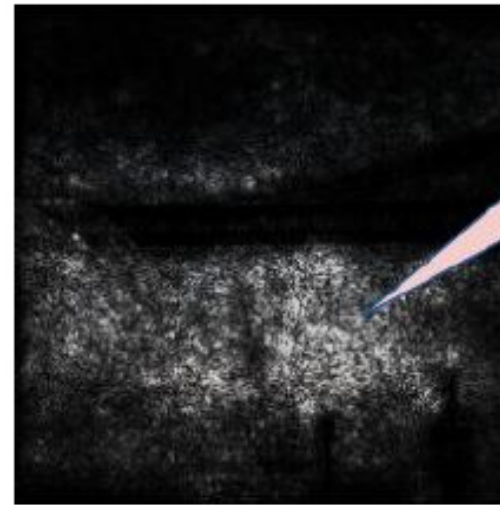
- Gradients can get saturated
- Solution: Take (element-wise) product of the gradient with the input itself
- Can produce visually simpler/sharper images
- Can be used in conjunction with any method

Gradient-Input



Integrated Gradients

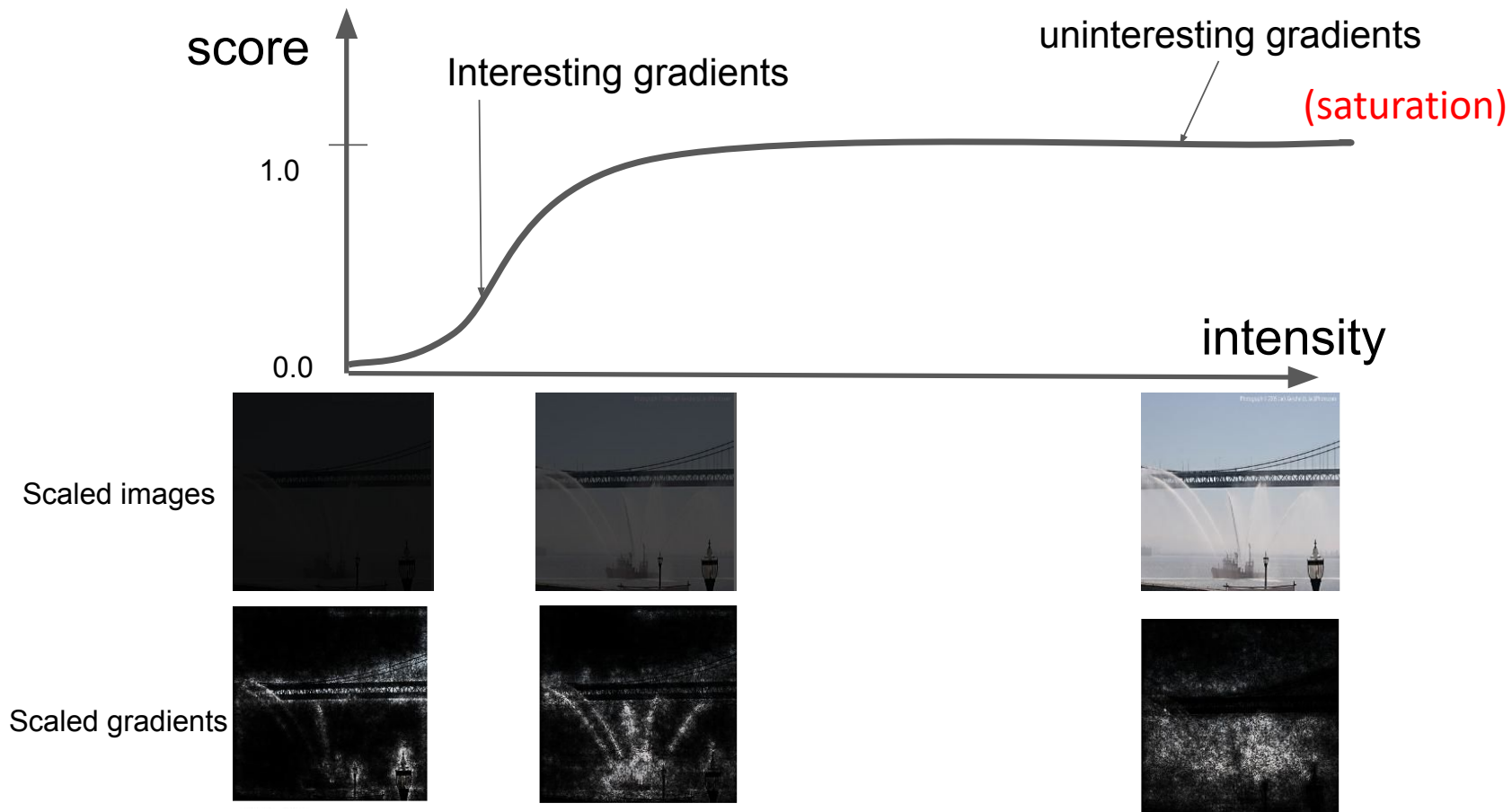
- Input gradients indicate influence
- However, gradient for actual input may not give the full picture



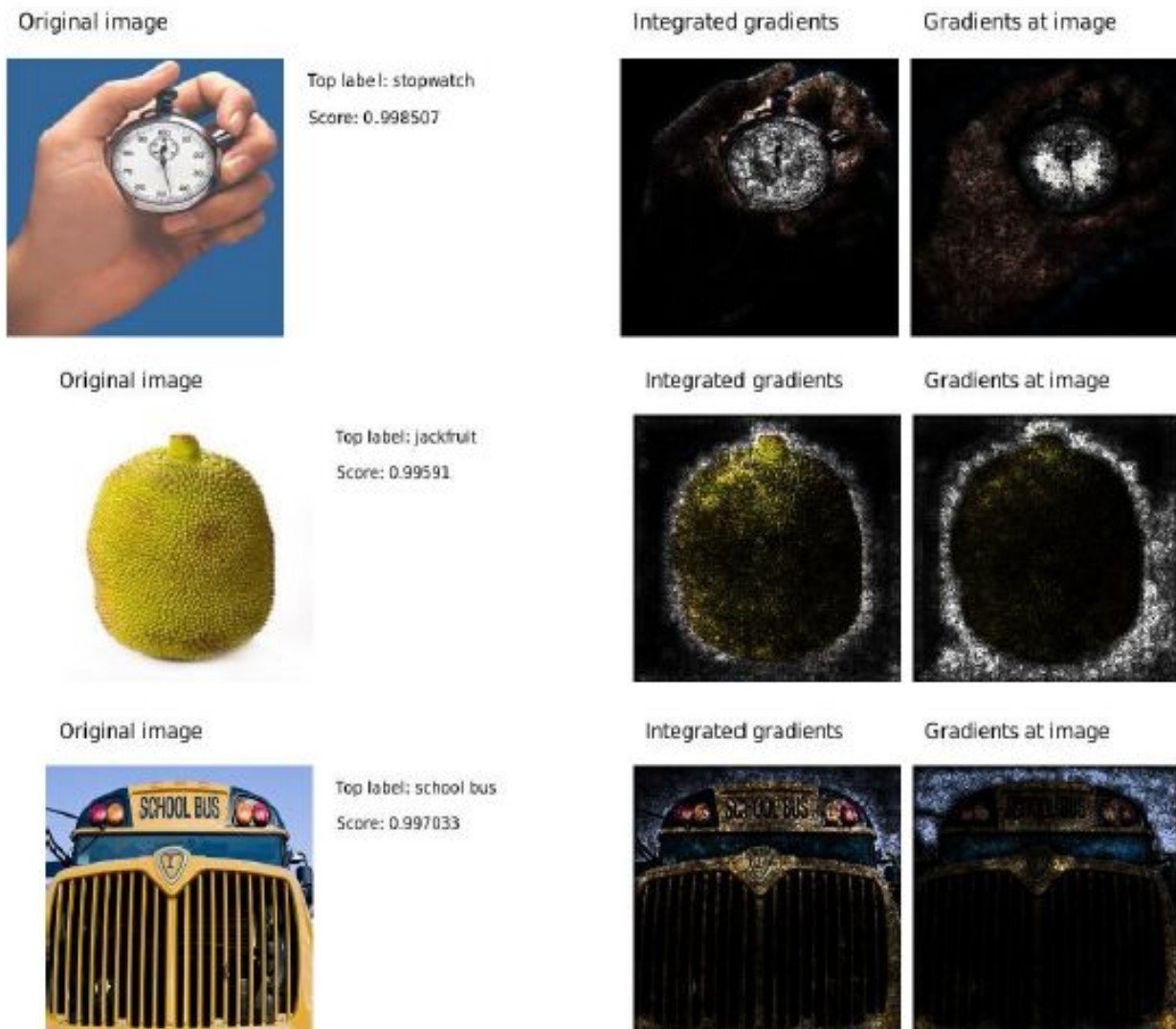
Gradients in the vicinity of the input seem like noise

Integrated Gradients

Solution: integrate gradients over a range of possible inputs between *baseline* input and given input



Examples of improved explanations



Integrated Gradients: Key ideas

- **Baseline input:** A baseline is an uninformative input used for comparison
 - E.g. Object recognition: black image, noise image
- List of **desirable criteria (axioms)** for an attribution method
 - **Implementation Invariance Axiom:** Two networks that compute identical functions for all inputs get identical attributions even if their architecture/parameters differ
 - **Sensitivity Axiom:**
 - If baseline and input have different scores, but differ in a single variable, then that variable gets some attribution.
 - If a variable has no influence on a function, then it gets no attribution.
 - **Linearity Preservation Axiom:** $\text{Attributions}(a*f1 + b*f2) = a*\text{Attributions}(f1) + b*\text{Attributions}(f2)$
 - **Completeness Axiom:** $\text{sum}(\text{attributions}) = f(\text{input}) - f(\text{baseline})$
 - **Symmetry Preservation Axiom:** Symmetric variables with identical values get equal attributions
- Integrated Gradients is the only method that satisfies these desirable criteria

Other popular approaches

- **SHAP:**
 - Model-agnostic approach to calculating feature importance
 - Does not require calculating gradients
 - Based on cooperative game-theoretic ideas
 - Lundberg et al., A Unified Approach to Interpreting Model Predictions, NeurIPS'17
- **Influence functions:**
 - Find the training samples most responsible for a particular prediction at a given test point
 - Gradient-based approach
 - Koh et al., Understanding Black-box Predictions via Influence Functions, ICML'17
- **Counterfactual explanations:**
 - Calculates smallest needed change to the input that would lead to a desirable outcome
 - Gradient-based approach
 - Wachter et al., Counterfactual explanations without opening the black box: Automated decisions and the GDPR, Harv. JL & Tech. 31 (2017)